

BIG DATA ANALYTICS

UNIT – II : Question Bank

Hadoop MapReduce | Apache Hive | HiveQL | Joins | Window Functions

SECTION A

Short Answer Questions (2–3 marks each)

Q1. What is Hadoop MapReduce and what are its two main tasks?

Answer:

Hadoop MapReduce is a programming framework for processing huge amounts of structured and unstructured data stored in HDFS. It processes data in parallel by dividing jobs into independent tasks, improving speed and reliability.

Two main tasks:

- Map Task – Loads, parses, transforms, and filters data. Converts input into intermediate key-value pairs.
- Reduce Task – Groups and aggregates the output produced by Map tasks to generate the final output.

Q2. Explain InputSplits in Hadoop MapReduce.

Answer:

An InputSplit is the logical representation of data processed by an individual Mapper. Key points:

- For each split, one Map task is created → number of map tasks = number of InputSplits.
- InputSplit does NOT contain actual data; it is only a reference to the data.
- InputFormat creates InputSplits and divides them into records.
- Users do not deal with InputSplits directly.

Example: A 500 MB file with a default block size of 128 MB creates 4 InputSplits, spawning 4 Map tasks.

Q3. What is the role of the Combiner in MapReduce?

Answer:

The Combiner (also called a semi-reducer or local reducer) works between the Mapper and Reducer to reduce network congestion.

- It performs local aggregation on the intermediate key-value pairs output by one mapper.
- Reduces the volume of data transferred over the network to the Reducer.
- Configured using: `Job.setCombinerClass(ReducerClass)`.

Example: If Mapper outputs (Hello,1),(Hello,1),(Hello,1), Combiner reduces it to (Hello,3) before sending to the Reducer.

Q4. Distinguish between Job Tracker and Task Tracker.

Answer:

Job Tracker (Master Daemon):

- Manages all resources and jobs across the cluster.
- Creates execution plans, assigns tasks to nodes.
- Monitors running tasks; re-schedules failed tasks.
- One Job Tracker per Hadoop cluster.

Task Tracker (Slave Daemon):

- Executes Map and Reduce tasks as instructed by Job Tracker.
- Sends periodic heartbeat signals to Job Tracker to indicate it is alive.
- Deployed on each node of the cluster.

Q5. What is Shuffling and Sorting in MapReduce?

Answer:

Shuffling: The process of transferring intermediate key-value pairs from Mappers to Reducers. It combines all values associated with the same key.

Example: (Blue,1),(Blue,1),(Blue,1) → Shuffle → (Blue,[1,1,1])

Sorting: After shuffling, key-value pairs are automatically sorted by key using the built-in WritableComparableInterface.

Together, shuffle and sort prepare organized input for the Reduce phase.

Q6. What is Apache Hive? List its main features.

Answer:

Apache Hive is a data warehouse tool built on top of Hadoop, used for querying and analyzing structured data using HiveQL (HQL) — an SQL-like language that gets converted internally to MapReduce jobs.

Originally developed by Facebook for processing ~20TB of data daily.

Key Features:

- Open-Source | • Supports multiple file formats (TextFile, ORC, Parquet)
- HiveQL similar to SQL | • Fast, scalable, extensible
- Supports DDL, DML, and User Defined Functions (UDFs)
- ETL Support | • Ad-hoc query execution

Q7. List the primitive data types supported by Hive.

Answer:

Hive primitive data types are categorized as:

- Numeric (Integral): TINYINT (1 byte), SMALLINT (2 bytes), INT (4 bytes), BIGINT (8 bytes)
- Numeric (Floating): FLOAT (4 bytes), DOUBLE (8 bytes), DECIMAL(p,s) e.g. DECIMAL(10,0)
- String: STRING (unlimited), VARCHAR (1–65535), CHAR (max 255)
- Date/Time: TIMESTAMP (nanosecond precision, YYYY-MM-DD hh:mm:ss), DATE (YYYY-MM-DD)
- Miscellaneous: BOOLEAN (true/false), BINARY (array of bytes)

Q8. What are the DDL commands supported in Hive? Give the syntax to create a database.

Answer:

HiveQL DDL commands: CREATE, DROP, TRUNCATE, ALTER, SHOW, DESCRIBE, USE.

Syntax to Create a Database:

```
CREATE DATABASE [IF NOT EXISTS] database_name
[COMMENT 'comment'] [LOCATION hdfs_path]
[WITH DBPROPERTIES (key='value')];
```

Example:

```
CREATE DATABASE IF NOT EXISTS studentdb
```

```
COMMENT 'Student records database';
```

To show all databases: SHOW DATABASES;

To drop a database: DROP DATABASE IF EXISTS studentdb CASCADE;

Q9. Explain Table Partitioning in Hive with an example.

Answer:

Partitioning splits a large table into smaller sub-parts based on one or more columns (partition key), making queries faster on specific data slices.

- Hive stores each partition as a sub-directory in HDFS.
- Directory name = partition key and value (e.g., dept=CSE).

Example — Creating a partitioned table:

```
CREATE TABLE student_part (name STRING, rollno INT)
```

```
PARTITIONED BY (dept STRING)
```

```
ROW FORMAT DELIMITED FIELDS TERMINATED BY ',';
```

```
Loading data: LOAD DATA LOCAL INPATH '/data/cse.csv'
```

```
INTO TABLE student_part PARTITION (dept='CSE');
```

Q10. What is Bucketing in Hive and how does it differ from Partitioning?

Answer:

Bucketing further divides partition data into fixed number of files (buckets) based on the hash function of a column.

Difference:

- Partitioning: splits data based on distinct column values (e.g., year=2023). Number of partitions = number of distinct values.
- Bucketing: splits data into a fixed number of files using $\text{hash}(\text{column}) \% \text{num_buckets}$. Enables efficient sampling and map-side joins.

Example:

```
CREATE TABLE student_bucket (name STRING, id INT)
```

```
CLUSTERED BY (id) INTO 4 BUCKETS
```

```
ROW FORMAT DELIMITED FIELDS TERMINATED BY ',';
```

SECTION B

Long Answer Questions (10–15 marks each)

Q1. Describe the complete working of Hadoop MapReduce with all its phases: Input Splits, Map Phase, Combiner, Shuffle, Sort, and Reduce Phase. Use a word count example to illustrate.

Answer:

1. Input Files & InputFormat

Data for a MapReduce job is stored in HDFS in arbitrary formats. InputFormat defines how input files are split and read. It creates InputSplits and uses RecordReader to convert each split into key-value pairs.

2. InputSplits

Each InputSplit is a logical reference to a chunk of data processed by one Mapper. For a 500 MB file with 128 MB block size → 4 InputSplits → 4 Map tasks are spawned.

3. RecordReader

RecordReader communicates with InputSplit and converts data into (key, value) pairs. By default, TextInputFormat is used → key = byte offset, value = line of text.

Example: (0, 'Dear Bear River') → sent to Mapper.

4. Map Phase (Mapper)

Mapper processes each input record and generates intermediate key-value pairs stored on local disk (not HDFS).

Word Count Example:

Input: 'Dear Bear River'

Output: (Dear,1),(Bear,1),(River,1)

5. Combiner (Local Reducer)

Combiner performs local aggregation on Mapper output to reduce network traffic.

Example: If two Mappers both output (Bear,1),(Bear,1) → Combiner reduces to (Bear,2) each before network transfer.

6. Partitioner

Partitioner determines which Reducer receives which key. Records with the same key go to the same partition/Reducer. Number of partitioners = number of Reducers.

7. Shuffle Phase

Transfers intermediate data from Mappers to Reducers. Groups all values for the same key.

Example: (Bear,1),(Bear,1),(Bear,1) → (Bear,[1,1,1])

8. Sort Phase

Output is automatically sorted by key using WritableComparableInterface.

(Bear,[1,1,1]), (Car,[1,1,1]), (Deer,[1,1]), (River,[1,1])

9. Reduce Phase (Reducer)

Reducer applies user-defined aggregation logic on each key's value list.

Word Count: (Bear,[1,1,1]) → (Bear,3)

Final output is written to HDFS via OutputFormat.

Result: Bear→2, Car→3, Deer→2, River→2

Q2. Explain the Architecture of Apache Hive in detail, covering Hive Clients, Hive Services (Driver, Compiler, MetaStore, Execution Engine), and Hive Storage. Also describe the step-by-step working of Hive.

Answer:

Hive Architecture Overview

Hive architecture consists of 3 core parts: (1) Hive Clients, (2) Hive Services, (3) Hive Storage and Computing.

1. Hive Clients

Hive supports three types of clients:

- Thrift Client – Cross-language service for programming languages supporting Thrift.
- JDBC Driver – Connects Hive with Java applications (class: org.apache.hadoop.hive.jdbc.HiveDriver).
- ODBC Driver – Allows ODBC-protocol applications to connect to Hive.

2. Hive Services

- CLI – Command Line Interface to run HiveQL queries.
- Web UI – Browser-based alternative to CLI.

- MetaStore – Central repository storing table structure, partitions, column types, serializers/deserializers, and HDFS file locations.
- HiveServer – Apache Thrift Server; accepts client requests and forwards to Driver.
- Driver – Receives queries and passes to Compiler.
- Compiler – Parses HiveQL, performs semantic analysis, converts to MapReduce job plan.
- Execution Engine – Executes the logical plan generated by Optimizer in dependency order.

3. Hive Storage and Computing

- Metadata is stored in the MetaStore database (e.g., MySQL, Derby).
- Query results and loaded table data are stored in HDFS in the Hadoop cluster.

Step-by-Step Working of Hive

- Step 1: Execute Query – CLI/Web UI sends query to Driver.
- Step 2: Get Plan – Driver uses Compiler to parse and create query plan.
- Step 3: Get Metadata – Compiler requests metadata from MetaStore.
- Step 4: Send Metadata – MetaStore returns table/column info to Compiler.
- Step 5: Send Plan – Compiler sends optimized logical plan back to Driver.
- Step 6: Execute Plan – Driver sends plan to Execution Engine.
- Step 7: Execute Job – Execution Engine submits MapReduce job to JobTracker → TaskTrackers execute Map and Reduce tasks.
- Step 8–10: Results are fetched from DataNodes → sent to Execution Engine → Driver → Hive Interface.

Q3. Explain HiveQL DDL and DML operations with complete syntax and examples. Cover: CREATE TABLE, LOAD DATA, INSERT, SELECT with WHERE/GROUP BY, UPDATE, and DELETE.

Answer:

DDL – CREATE TABLE

Syntax:

```
CREATE TABLE [IF NOT EXISTS] table_name (col1 datatype, col2 datatype)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS TEXTFILE;
```

```
CREATE TABLE IF NOT EXISTS student (
    Student_Name STRING,
    Student_Rollno INT,
    Student_Marks FLOAT)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ',';
```

DML – LOAD DATA

Used to load data from local filesystem or HDFS into a Hive table.

Syntax: LOAD DATA [LOCAL] INPATH '<path>' [OVERWRITE] INTO TABLE <table>;

```
-- From local filesystem:
LOAD DATA LOCAL INPATH '/home/user/data.csv'
INTO TABLE student;

-- From HDFS (omit LOCAL):
LOAD DATA INPATH '/hdfs/data/student.csv'
INTO TABLE student;
```

DML – INSERT

Used to insert records directly into a Hive table.

```
INSERT INTO TABLE student VALUES
```

```
('Dikshant', 1, 95),
('Akshat', 2, 96),
('Dhruv', 3, 90);
```

DML – SELECT

Retrieves data from tables. Supports WHERE, GROUP BY, ORDER BY, LIMIT.

```
-- Basic select:
SELECT * FROM student;

-- With WHERE clause:
SELECT Student_Name, Student_Marks
FROM student WHERE Student_Marks > 90;

-- With GROUP BY:
SELECT dept, COUNT(*) AS total
FROM student GROUP BY dept;
```

DML – UPDATE and DELETE

UPDATE: Modifies existing records. Cannot be used on Partitioning or Bucketing columns.

DELETE: Removes rows matching a condition.

```
-- UPDATE:
UPDATE student SET Student_Marks = 98
WHERE Student_Rollno = 1;

-- DELETE:
DELETE FROM student
WHERE Student_Marks < 50;
```

Q4. Explain Logical Joins in Hive with syntax and examples. Cover Inner Join, Left Outer Join, Right Outer Join, and Full Outer Join using the CUSTOMERS and ORDERS tables.

Answer:

Overview of Hive Joins

Hive JOINS combine columns from two or more tables using common values. Important rules:

- Only Equality joins are allowed in Hive.
- More than two tables can be joined in one query.
- Hive Joins are NOT commutative.

Tables used: CUSTOMERS (ID, Name, Age, Address, Salary) and ORDERS (OID, Date, Customer_ID, Amount).

1. INNER JOIN

Returns records that have matching values in BOTH tables.

```
SELECT c.ID, c.NAME, c.AGE, o.AMOUNT
FROM CUSTOMERS c JOIN ORDERS o
ON (c.ID = o.CUSTOMER_ID);

-- Result (only matching rows):
-- 3  Chandler  23  3000
-- 3  Chandler  23  1500
-- 2  Rachel   25  1560
-- 4  Monika   25  2060
```

2. LEFT OUTER JOIN

Returns ALL rows from the LEFT table and matching rows from the right table. Non-matching right columns are NULL.

```
SELECT c.ID, c.NAME, o.AMOUNT, o.DATE
FROM CUSTOMERS c LEFT OUTER JOIN ORDERS o
ON (c.ID = o.CUSTOMER_ID);

-- Result includes all 7 customers;
-- Ross, Mike, Phoebe, Joey get NULL for Amount and Date.
```

3. RIGHT OUTER JOIN

Returns ALL rows from the RIGHT table and matching rows from the left table. Non-matching left columns are NULL.

```
SELECT c.ID, c.NAME, o.AMOUNT, o.DATE
FROM CUSTOMERS c RIGHT OUTER JOIN ORDERS o
ON (c.ID = o.CUSTOMER_ID);

-- Result includes all 4 orders;
-- Customers with no orders not included.
```

4. FULL OUTER JOIN

Returns ALL rows from BOTH tables. Non-matching columns on either side are filled with NULL.

```
SELECT c.ID, c.NAME, o.AMOUNT, o.DATE
FROM CUSTOMERS c FULL OUTER JOIN ORDERS o
ON (c.ID = o.CUSTOMER_ID);

-- Result: All 7 customers + all 4 orders.
-- Customers without orders: NULL in Amount, Date.
-- Orders without customers: NULL in ID, Name.
```

Q5. Explain Window Functions in Hive in detail. Cover PARTITION BY, ORDER BY, LEAD, LAG, FIRST_VALUE, and LAST_VALUE with examples using the employee table.

Answer:

Window Functions Overview

A window function performs calculations across a set of table rows without collapsing them (unlike GROUP BY). General syntax:

```
SELECT col, AGG_FUNC(col) OVER (PARTITION BY col ORDER BY col) FROM table;
```

Two types: (1) Aggregate Functions: COUNT, SUM, MIN, MAX, AVG.

(2) Analytical Functions: LEAD, LAG, FIRST_VALUE, LAST_VALUE.

PARTITION BY

Groups rows into partitions for the window function, similar to GROUP BY but without collapsing rows.

```
-- Count employees in each department:
SELECT department, COUNT(id)
OVER (PARTITION BY department)
FROM emp_dept_tbl;

-- Output: Each row shows total count for its dept.
-- Account dept: 5 | Sales dept: 5
```

ORDER BY (with and without PARTITION)

Controls the ordering within each partition window.

```
-- Without PARTITION (cumulative count by salary):
SELECT id, salary, COUNT(id)
OVER (ORDER BY salary DESC)
FROM emp_dept_tbl;

-- With PARTITION (count within dept by salary):
```

```
SELECT id, department, salary, COUNT(id)
OVER (PARTITION BY department ORDER BY salary DESC)
FROM emp_dept_tbl;
```

LEAD

Returns value from the NEXT row in the window. Returns NULL if no next row exists.

```
SELECT id, first_name, salary,
       LEAD(id) OVER
       (PARTITION BY department ORDER BY salary)
FROM emp_dept_tbl;
-- For last row in each dept: LEAD = NULL
-- Example: id=1008 (Manager, salary=28000) → LEAD=NULL
```

LAG

Opposite of LEAD — returns value from the PREVIOUS row. Returns NULL for the first row in the window.

```
SELECT id, first_name, salary,
       LAG(id) OVER
       (PARTITION BY department ORDER BY salary)
FROM emp_dept_tbl;
-- First row of each dept: LAG = NULL
-- Example: id=1009 (salary=12000, first in Account) → LAG=NULL
```

FIRST_VALUE and LAST_VALUE

FIRST_VALUE returns the value from the first row of the window partition, assigned to all rows in that group.

LAST_VALUE returns the value from the last row seen so far (based on current frame).

```
-- FIRST_VALUE:
SELECT id, department, salary,
       FIRST_VALUE(id)
       OVER (PARTITION BY department ORDER BY salary)
FROM emp_dept_tbl;
-- All rows in Account dept get 1009 (lowest salary)
-- All rows in Sales dept get 1010

-- LAST_VALUE:
SELECT id, department, salary,
       LAST_VALUE(id)
       OVER (PARTITION BY department ORDER BY salary)
FROM emp_dept_tbl;
```